

Лабораторная работа №1

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 FullName Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	6
3.1.2.1 operator!=(())	6
3.1.2.2 operator<()	6
3.1.2.3 operator<=()	6
3.1.2.4 operator==(())	7
3.1.2.5 operator>()	7
3.1.2.6 operator>=()	8
3.2 Sportsman Struct Reference	8
3.2.1 Detailed Description	9
3.2.2 Constructor & Destructor Documentation	9
3.2.2.1 Sportsman()	9
3.2.3 Member Function Documentation	10
3.2.3.1 operator!=(())	10
3.2.3.2 operator<()	10
3.2.3.3 operator<=()	10
3.2.3.4 operator==(())	11
3.2.3.5 operator>()	11
3.2.3.6 operator>=()	12
3.2.3.7 to_csv()	12
3.2.4 Member Data Documentation	12
3.2.4.1 age	12
3.2.4.2 full_name	12
3.2.4.3 height_cm	13
3.2.4.4 sport	13
3.2.4.5 weight_kg	13
4 File Documentation	15
4.1 src/sportsman.h File Reference	15
4.1.1 Detailed Description	15
4.2 sportsman.h	15
Index	19

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FullName	ФИО спортсмена	5
Sportsman	Структура данных, описывающая спортсмена	8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/sportsman.h	
Структура данных, описывающая спортсмена	15

Chapter 3

Class Documentation

3.1 FullName Struct Reference

ФИО спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- bool `operator==` (const `FullName` &other) const
Проверяет равенство двух ФИО
- bool `operator<` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator!=` (const `FullName` &other) const
Проверяет неравенство двух ФИО
- bool `operator>` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator<=` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator>=` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Public Attributes

- `std::string last_name`
- `std::string first_name`
- `std::string middle_name`

3.1.1 Detailed Description

ФИО спортсмена

3.1.2 Member Function Documentation

3.1.2.1 operator!=(())

```
bool FullName::operator!=(  
    const FullName & other) const [inline]
```

Проверяет неравенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО не совпадают, иначе false

3.1.2.2 operator<()

```
bool FullName::operator<(  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти раньше другого, иначе false

3.1.2.3 operator<=()

```
bool FullName::operator<= (  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не позже другого, иначе false

3.1.2.4 operator==()

```
bool FullName::operator== (
    const FullName & other) const [inline]
```

Проверяет равенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО совпадают, иначе false

3.1.2.5 operator>()

```
bool FullName::operator> (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти позже другого, иначе false

3.1.2.6 operator>=()

```
bool FullName::operator>= (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не раньше другого, иначе false

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

3.2 Sportsman Struct Reference

Структура данных, описывающая спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- [Sportsman](#) (std::string csv_string)
Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- std::string [to_csv](#) () const
Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- bool [operator==](#) (const [Sportsman](#) &other) const
Проверяет равенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator<](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator!=](#) (const [Sportsman](#) &other) const
Проверяет неравенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator>](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator<=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator>=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Public Attributes

- [FullName](#) `full_name`
- unsigned short [age](#)
- unsigned short [height_cm](#)
- unsigned short [weight_kg](#)
- `std::string` [sport](#)

3.2.1 Detailed Description

Структура данных, описывающая спортсмена

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Sportsman()

```
Sportsman::Sportsman (  
    std::string csv_string) [inline]
```

Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Parameters

<code>[csv_string]</code>	Строка в формате CSV
---------------------------	----------------------

Returns

Инициализированный объект структуры [Sportsman](#)

Exceptions

<code>std::invalid_argument</code> , если	строка не является корректной CSV строкой или содержит некорректные данные
---	--

3.2.3 Member Function Documentation

3.2.3.1 operator!=(())

```
bool Sportsman::operator!=(
    const Sportsman & other) const [inline]
```

Проверяет неравенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔	Спортсмен для сравнения
[other]↔	

Returns

true, если спортсмены не совпадают, иначе false

3.2.3.2 operator<()

```
bool Sportsman::operator< (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔	Спортсмен для сравнения
[other]↔	

Returns

true, если текущий спортсмен должен идти раньше другого, иначе false

3.2.3.3 operator<=()

```
bool Sportsman::operator<= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если текущий спортсмен должен идти не позже другого, иначе false

3.2.3.4 operator==()

```
bool Sportsman::operator== (  
    const Sportsman & other) const [inline]
```

Проверяет равенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если спортсмены совпадают, иначе false

3.2.3.5 operator>()

```
bool Sportsman::operator> (  
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если текущий спортсмен должен идти позже другого, иначе false

3.2.3.6 operator>=()

```
bool Sportsman::operator>= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

	Спортсмен для сравнения
[other] 	

Returns

true, если текущий спортсмен должен идти не раньше другого, иначе false

3.2.3.7 to_csv()

```
std::string Sportsman::to_csv () const [inline]
```

Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Returns

Строка в формате CSV

3.2.4 Member Data Documentation

3.2.4.1 age

```
unsigned short Sportsman::age
```

Возраст спортсмена в годах

3.2.4.2 full_name

```
FullName Sportsman::full_name
```

ФИО спортсмена

3.2.4.3 height_cm

```
unsigned short Sportsman::height_cm
```

Рост спортсмена в сантиметрах

3.2.4.4 sport

```
std::string Sportsman::sport
```

Спорт, которым занимается спортсмен

3.2.4.5 weight_kg

```
unsigned short Sportsman::weight_kg
```

Вес спортсмена в килограммах

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

Chapter 4

File Documentation

4.1 src/sportsman.h File Reference

Структура данных, описывающая спортсмена

```
#include <string>
```

Classes

- struct [FullName](#)
ФИО спортсмена
- struct [Sportsman](#)
Структура данных, описывающая спортсмена

4.1.1 Detailed Description

Структура данных, описывающая спортсмена

4.2 sportsman.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <string>
00009
00014 struct FullName
00015 {
00016     std::string last_name;    /**< Фамилия */
00017     std::string first_name;   /**< Имя */
00018     std::string middle_name;  /**< Отчество */
00019
00025     bool operator==(const FullName &other) const
00026     {
```

```

00027         return first_name == other.first_name &&
00028                middle_name == other.middle_name &&
00029                last_name == other.last_name;
00030     }
00031
00032     bool operator<(const FullName &other) const
00033     {
00034         if (last_name != other.last_name)
00035         {
00036             return last_name < other.last_name;
00037         }
00038         if (first_name != other.first_name)
00039         {
00040             return first_name < other.first_name;
00041         }
00042         return middle_name < other.middle_name;
00043     }
00044
00045     bool operator!=(const FullName &other) const
00046     {
00047         return !(*this == other);
00048     }
00049
00050     bool operator>(const FullName &other) const
00051     {
00052         return other < *this;
00053     }
00054
00055     bool operator<=(const FullName &other) const
00056     {
00057         return !(other < *this);
00058     }
00059
00060     bool operator>=(const FullName &other) const
00061     {
00062         return !(*this < other);
00063     }
00064 };
00065
00066 struct Sportsman
00067 {
00068     FullName full_name;
00069     unsigned short age;
00070     unsigned short height_cm;
00071     unsigned short weight_kg;
00072     std::string sport;
00073
00074     Sportsman(std::string csv_string)
00075     {
00076         size_t pos = 0;
00077         std::string token;
00078         std::string delimiter = ",";
00079         int field_index = 0;
00080
00081         for (int field_idx = 0; field_idx < 7; field_idx++)
00082         {
00083             pos = csv_string.find(delimiter);
00084             if (pos == std::string::npos)
00085             {
00086                 if (field_idx != 6)
00087                 {
00088                     throw std::invalid_argument("Недостаточно полей в CSV строке");
00089                 }
00090                 token = csv_string;
00091             }
00092             else if (field_idx == 6)
00093             {
00094                 throw std::invalid_argument("Слишком много полей в CSV строке");
00095             }
00096             else
00097             {
00098                 token = csv_string.substr(0, pos);
00099                 csv_string.erase(0, pos + delimiter.length());
00100             }
00101
00102             switch (field_idx)
00103             {
00104             case 0:
00105                 sport = token;
00106                 break;
00107             case 1:
00108                 full_name.last_name = token;
00109                 break;
00110             case 2:
00111                 full_name.first_name = token;
00112                 break;
00113             case 3:

```

```

00149         full_name.middle_name = token;
00150         break;
00151     case 4:
00152         age = (unsigned short)std::stoul(token);
00153         break;
00154     case 5:
00155         height_cm = (unsigned short)std::stoul(token);
00156         break;
00157     case 6:
00158         weight_kg = (unsigned short)std::stoul(token);
00159         break;
00160     }
00161 }
00162 }
00163
00164 std::string to_csv() const
00165 {
00166     return sport + "," +
00167         full_name.last_name + "," +
00168         full_name.first_name + "," +
00169         full_name.middle_name + "," +
00170         std::to_string(age) + "," +
00171         std::to_string(height_cm) + "," +
00172         std::to_string(weight_kg);
00173 }
00174
00175 bool operator==(const Sportsman &other) const
00176 {
00177     return sport == other.sport &&
00178         full_name == other.full_name &&
00179         age == other.age;
00180 }
00181
00182 bool operator<(const Sportsman &other) const
00183 {
00184     if (sport != other.sport)
00185     {
00186         return sport < other.sport;
00187     }
00188     if (full_name != other.full_name)
00189     {
00190         return full_name < other.full_name;
00191     }
00192     return age < other.age;
00193 }
00194
00195 bool operator!=(const Sportsman &other) const
00196 {
00197     return !(*this == other);
00198 }
00199
00200 bool operator>(const Sportsman &other) const
00201 {
00202     return other < *this;
00203 }
00204
00205 bool operator<=(const Sportsman &other) const
00206 {
00207     return !(other < *this);
00208 }
00209
00210 bool operator>=(const Sportsman &other) const
00211 {
00212     return !(*this < other);
00213 }
00214 };

```


Index

- age
 - Sportsman, [12](#)
- full_name
 - Sportsman, [12](#)
- FullName, [5](#)
 - operator!=, [6](#)
 - operator<, [6](#)
 - operator<=, [6](#)
 - operator>, [7](#)
 - operator>=, [7](#)
 - operator==, [7](#)
- height_cm
 - Sportsman, [12](#)
- operator!=
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator<
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator<=
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator>
 - FullName, [7](#)
 - Sportsman, [11](#)
- operator>=
 - FullName, [7](#)
 - Sportsman, [11](#)
- operator==
 - FullName, [7](#)
 - Sportsman, [11](#)
- sport
 - Sportsman, [13](#)
- Sportsman, [8](#)
 - age, [12](#)
 - full_name, [12](#)
 - height_cm, [12](#)
 - operator!=, [10](#)
 - operator<, [10](#)
 - operator<=, [10](#)
 - operator>, [11](#)
 - operator>=, [11](#)
 - operator==, [11](#)
 - sport, [13](#)
 - Sportsman, [9](#)
 - to_csv, [12](#)
 - weight_kg, [13](#)
 - src/sportsman.h, [15](#)
- to_csv
 - Sportsman, [12](#)
- weight_kg
 - Sportsman, [13](#)